Measuring an Asynchronous Report API

Laboratory 2
CMPS4191: Advanced Web Technologies
University of Belize

Andres Hung & Jennessa Sierra

https://github.com/jennxsierra/cmps4191-lab-02-asynchronous-api

Aug 28, 2026

LABORATORY 2		
	Title	Measuring an Asynchronous Report API
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Aug 28, 2026

Table of Contents

Table of Contents **2**
1) Experimental Design **3**
Research Question 3
Hypothesis 3
Variables 3
Measurement Criteria 3
Predictions 4
Contract Reconstruction 4
2) Procedure **5**
3) Results **6**
Raw Measurements 6
Condition: 0 seconds 6
Condition: 3 seconds 6
Condition: 7 seconds 6
Condition: 12 seconds 7
Summary 7
Laboratory 1 Comparison 7
4) Observations **8**
Explanation 8
5) Contract Evaluation **9**
6) Limitations & New Questions **9**
7) The Next Experiment **10**
Appendices **11**
Appendix A - Supplied Script 11

LABORATORY 2		
	Title	Measuring an Asynchronous Report API
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Aug 28, 2026

1) Experimental Design

Research Question

How does artificial report-generation delay affect (1) POST acknowledgment time and (2) total job-completion time in the asynchronous API?

Hypothesis

If report-generation delay increases, then acknowledgment time will not increase, while completion time will increase, because completion time depends on report-generation delay, while acknowledgment time does not.

Variables

Role	Variable
Independent	Artificial worker delay: 0s, 3s, 7s, 12s
Dependent	POST acknowledgment time, total completion time, poll count, observed states, and final state
Controlled	Endpoint, payload, consumer, database, worker count, polling interval, client script, and server configuration

Table 1.1: List of independent, dependent, and controlled variables.

Measurement Criteria

Measure	Start	Stop	What it tests
T_ack	Immediately before POST	A 202 response received	Whether the submission waits for the report work
T_complete	Immediately before POST	Completed or failed observed	How long does the entire operation take

Table 1.2: Description of the measurements to be taken for the lab and their criteria.

LABORATORY 2		
	Title	Measuring an Asynchronous Report API
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Aug 28, 2026

Predictions

Delay	Predicted T_ack	Predicted T_complete	Expected Final State
0s	0.005s	0.01s	Completed
3s	0.005s	3.01s	Completed
7s	0.005s	7.01s	Completed
12s	0.005s	12.01s	Completed

Table 1.3: Predictions of the T_ack time, T_complete time, and expected final state for each report-generation delay.

Contract Reconstruction

What fact does 202 Accepted communicate?
A 202 Accepted HTTP response status code, sent by the responding server, communicates that the request was accepted for processing but that processing has not yet begun or been completed.

What fact does it not communicate?
It does not indicate whether the processing directed by the request succeeds or fails. The processing done by the server is not guaranteed. It may fail or be disallowed.

Why must the server return a job identifier?
Because the initial job request is completed with the 202 Accepted response, the client needs a way to identify the job for querying. The job identifier that the server returns is used for that.

Which request retrieves the eventual outcome?
One of the HTTP GET requests the client sends to poll the job's status will return the final outcome (completed or failed).

Which component performs the report work?
The worker component performs the report work. This component is typically another program or another thread of a program that runs concurrently from the main API server program. Implementation options include a Python program or a Goroutine.

LABORATORY 2	Title	Measuring an Asynchronous Report API
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Aug 28, 2026

2) Procedure

1. The supplied system was verified for the following:
    a. The POST handler created a queued job and immediately returned a 202 Accepted response.
    b. The worker applied the configured artificial delay, generated the report, and recorded the result.
    c. The GET **/v1/jobs/{id}** endpoint returned the job's current state.
    d. The measurement script polled every 250ms until it completed or failed.
2. The following request.json was created.

```
{
  "consumer_id": "0198f000-0000-7000-8000-000000000001",
  "from": "2026-01-01T00:00:00Z",
  "to": "2027-01-01T00:00:00Z"
}
```

3. The supplied system was started with the following command, and the command was later repeated for report-delay parameters of 3s, 7s, and 12s.

```
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=0s
```

4. The supplied script was saved as **measure_async.sh**, and executed. The script records the start time before sending the initial HTTP request to the **/v1/reports** endpoint to generate the report. Then, the acknowledgment time (T_ack) is recorded, and both it and the job ID are printed. Afterward, the job status and poll count are continuously polled every 0.25s until the job status is completed or failed. Lastly, the end time is recorded; the complete time (T_complete) is calculated by subtracting the end time from the start time; and a summary of the job and measurements is printed. The following command was run to make the script executable. The supplied script can be found in Appendix A.

```
chmod +x measure_async.sh
./measure_async.sh
```

5. The script was run three times for each trial to measure the current delay.
6. For each run, T_ack, T_complete, poll count, and final state were recorded.
7. At least one GET response in each condition was inspected.
8. The server was restarted with the next delay between each trial.
9. Different delay conditions were not run simultaneously.

LABORATORY 2		
	Title	Measuring an Asynchronous Report API
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Aug 28, 2026

3) Results

Raw Measurements

Condition: 0 seconds

Run	T_ack (s)	T_complete (s)	Polls	Final State	States Observed
1	0.004693	0.046	1	completed	queued, completed
2	0.001919	0.324	2	completed	queued, completed
3	0.001759	0.325	2	completed	queued, completed

Table 3.1: Raw measurement results for the API server run with 0s delay report generation.

Condition: 3 seconds

Run	T_ack (s)	T_complete (s)	Polls	Final State	States Observed
1	0.005399	3.176	12	completed	queued, processing, completed
2	0.001910	3.147	12	completed	queued, processing, completed
3	0.001667	3.442	13	completed	queued, processing, completed

Table 3.2: Raw measurement results for the API server run with 3s delay report generation.

Condition: 7 seconds

Run	T_ack (s)	T_complete (s)	Polls	Final State	States Observed
1	0.004846	7.437	27	completed	queued, processing, completed
2	0.001715	7.414	27	completed	queued, processing, completed
3	0.002200	7.394	27	completed	queued, processing, completed

Table 3.3: Raw measurement results for the API server run with 7s delay report generation.

LABORATORY 2		
	Title	Measuring an Asynchronous Report API
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Aug 28, 2026

Condition: 12 seconds

Run	T_ack (s)	T_complete (s)	Polls	Final State	States Observed
1	0.005745	12.215	44	completed	queued, processing, completed
2	0.001779	12.487	45	completed	queued, processing, completed
3	0.002175	12.207	44	completed	queued, processing, completed

Table 3.4: Raw measurement results for the API server run with 12s delay report generation.

Summary

Delay	Mean T_ack	Mean T_complete	Successful Jobs
0s	0.002790s	0.231667s	3 / 3
3s	0.002992s	3.255000s	3 / 3
7s	0.002920s	7.415000s	3 / 3
12s	0.003233s	12.303000s	3 / 3

Table 3.5: Calculated Mean Time results for T_ack and T_complete varying by delay. Note: Mean Time was calculated by the formula: $(T_1 + T_2 + T_3) / 3$, where T is a recorded time. Results were rounded to the 6th decimal point.

Laboratory 1 Comparison

Delay	Synchronous Response Time	Async T_ack (s)	Async T_complete (s)
0s	0.008188s	0.002790	0.231667
3s	3.005672s	0.002992	3.255000
7s	7.005822s	0.002920	7.415000
12s	12.002793s	0.003233	12.303000

Table 3.6: Time results comparison with Laboratory 1.

LABORATORY 2		
	Title	Measuring an Asynchronous Report API
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Aug 28, 2026

4) Observations

The measurements for T_ack remained consistent and low across all conditions. Comparing the mean asynchronous T_ack and the mean synchronous response time from Laboratory 1, based on the first response sent by the server, T_ack remained low throughout, while the synchronous response time increased with delay. The mean asynchronous T_complete followed the same trend as the synchronous response time, increasing with increasing delay. There was a relatively noticeable increase of approximately 8% in overall time in T_complete compared to synchronous response time and the predictions. Poll counts between trials of the same condition sometimes differed by 1.

Explanation

The approximate 8% increase in T_complete time relative to the synchronous response time can be attributed to the additional delay introduced by the polling interval between polling requests made by the measurement script, as well as the time interval between job creation and worker claim. Both were configured to 0.25s, which is about the 8% increase seen in the observations. This reasoning can also explain poll counts differing by 1 between trials of the same condition.

Why did T_ack behave differently from synchronous response time?
T_ack behaved differently from the synchronous response time because the API server code was changed to handle the initial request asynchronously. Instead of the **createReportHandler** processing the report before sending a response, it now submits a job for another process to handle report generation. This is comparatively faster since T_ack is independent of the report generation time (report delay). A response is sent even before the job is finished.

Why did T_complete still change with artificial delay?
T_complete still changed with the artificial delay since job completion is still dependent on report generation being complete. A job is not considered complete until the report generation has finished.

Which component now owns the long-running work?
The worker component now owns the long-running work. This was implemented with a goroutine created in the **startReportWorker** method, which was started when the API server was started.

What evidence shows that the original request ended before the report was completed?
The fact that the **measure_async.sh** script was able to poll (send GET requests) the status of the job after its original request (POST request) shows that the original request ended before the report was completed. This is evident in the “status=processing” logs preceding the “status=completed” log for the script.

LABORATORY 2	Title	Measuring an Asynchronous Report API
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Aug 28, 2026

5) Contract Evaluation

Requirement	Evidence	Supported?
Acknowledge without waiting for report completion	Mean T_ack remained between 0.002790s and 0.003233s in all trials of increasing artificial delay.	Yes
Allow work to continue after POST ends	T_complete was longer than T_ack, which represented the point after which the initial POST request was completed, according to the measurement script. Later polls from the script observed the job progressing through queued, processing, and completed states.	Yes
Expose current job state	The measurement script polling requests for GET `/v1/jobs/{id}` returned observable queued, processing, and completed states.	Yes
Exposes completed result or failure	Every run exposed a completed terminal state, but no failed job was produced to verify the failure response.	Unclear

Table 5.1: Evaluation of the contract and determinations of whether the requirements are supported by the evidence.

6) Limitations & New Questions

This experiment includes some limitations. Some properties were not tested, such as:

- Worker-crash recovery: The experiment did not test whether a queued or processing job would survive and resume after the worker or API server crashed. A new question is whether the job would be retried, fail, or remain stuck.
- Failure handling: All 12 jobs completed successfully, so the experiment did not verify how the API exposes a failed job or its error message. A new question is whether clients receive a clear and useful failure response through GET `/v1/jobs/{id}`.
- Duplicate submissions: For this experiment, duplicate submissions, as done between trials of the all condition, were treated as separate jobs. A new question would be how duplicate submissions can be properly recognized and handled.

LABORATORY 2		
	Title	Measuring an Asynchronous Report API
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Aug 28, 2026

7) The Next Experiment

The asynchronous contract successfully decouples acknowledgment time (T_ack) from report-generation duration. Mean T_ack increased only slightly from 0.002790s at a 0s delay to 0.003233s at a 12s delay, whereas mean T_complete increased from 0.231667s to 12.303s. This pattern occurred because the POST handler returned a `202 Accepted` response after creating the job, while the worker generated the report independently. However, the experiment cannot conclude that the system recovers correctly from worker crashes or handles failed jobs properly.

As the artificial work duration increased, T_ack remained nearly constant while T_complete increased with the configured delay. This tested the requirement that the client regain control without waiting for the report to be completed. POST `/v1/reports` promised only that the work was accepted and returned a stable job identifier, whereas GET `/v1/jobs/{id}` promised to expose the job's current state and eventual result. The POST handler created a queued job, the worker performed the long-running report generation, and the client polled the GET endpoint until a terminal state was observed. The measurements therefore support the contract claim that acknowledgment is independent of report duration, although asynchronous execution did not make the report itself finish sooner.

The next experiment should test worker-crash recovery. A report job should be submitted with a long artificial delay, such as 12s, and the worker should be stopped after the job enters the processing state. The API and worker should then be restarted while the client continues polling the same job identifier. The experiment should record whether the job is retried, completed, marked as failed, or left permanently in processing. Repeating this test three times would provide evidence about whether accepted jobs survive worker failures and whether the GET endpoint exposes a clear terminal outcome.

LABORATORY 2		
	Title	Measuring an Asynchronous Report API
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Aug 28, 2026

Appendices

Appendix A - Supplied Script

```
#!/usr/bin/env bash
set -u

api='http://localhost:4000/v1'
start_ns=$(date +%s%N)

submit=$(curl --silent --show-error \
  --write-out $'\n%{time_total}' \
  --request POST \
  --header 'Content-Type: application/json' \
  --data @request.json \
  "$api/reports")

ack_time=${submit##*$'\n'}
body=${submit%$'\n'*}
job_id=$(printf '%s' "$body" | jq -r '.job_id')
polls=0

while true; do
  job=$(curl --silent --show-error "$api/jobs/$job_id")
  status=$(printf '%s' "$job" | jq -r '.job.status')
  polls=$((polls + 1))
  printf 'poll=%d status=%s\n' "$polls" "$status"
  case "$status" in completed|failed) break ;; esac
  sleep 0.25
done

end_ns=$(date +%s%N)
complete_time=$(awk -v s="$start_ns" -v e="$end_ns" \
  'BEGIN { printf "%.3f", (e-s)/1000000000 }')
printf 'job_id=%s\nack_time=%ss\ncomplete_time=%ss\npolls=%d\nfinal=%s\n' \
  "$job_id" "$ack_time" "$complete_time" "$polls" "$status"
```